

Guía de Referencia

MIPS32 - Arquitectura y Conjunto de Instrucciones

Estudiante de Arquitectura de Computadores

18 de mayo de 2026

Índice

1. Introducción a MIPS32	2
1.1. Características Principales	2
2. Convención de Notaciones	2
3. Registros del MIPS32	2
3.1. Convenciones de Uso de Registros	3
4. Conjunto de Instrucciones	3
4.1. Instrucciones Aritméticas	3
4.2. Instrucciones Lógicas	4
4.3. Instrucciones de Desplazamiento	4
4.4. Instrucciones de Transferencia de Datos	5
4.5. Instrucciones de Comparación y Salto	5
4.6. Instrucciones de Control del Sistema - Syscall	6
5. Estructura de un Programa en Ensamblador MIPS	7
5.1. Directivas de Datos más Comunes	7
6. Pseudoinstrucciones	7
7. Modos de Direccionamiento	8
8. Ejemplos Prácticos	8
8.1. Suma de dos números ingresados por el usuario	8
8.2. Bucle for para sumar números del 1 al N	9
9. Referencias	10

1. Introducción a MIPS32

MIPS (Microprocessor without Interlocked Pipeline Stages) es una arquitectura de tipo RISC (Reduced Instruction Set Computer) desarrollada originalmente por John L. Hennessy en la Universidad de Stanford. El libro "Computer Organization and Design" de Patterson y Hennessy utiliza MIPS como arquitectura de estudio debido a su elegancia y simplicidad [1].

1.1. Características Principales

- Conjunto de instrucciones reducido y ortogonal
- Formato de instrucciones de 32 bits de longitud fija
- 32 registros de propósito general de 32 bits
- Modos de direccionamiento simples
- Ideal para la enseñanza de arquitectura de computadores

2. Convención de Notaciones

En este documento se utilizarán las siguientes notaciones estándar para describir las instrucciones MIPS [2]:

Notación	Descripción
Rd, Rs, Rt	Registros (destino, fuente1, fuente2)
Imm16	Valor inmediato de 16 bits
Label	Etiqueta que representa una dirección de memoria
Offset(Rb)	Direccionamiento base+desplazamiento
# comentario	Comentario en ensamblador

3. Registros del MIPS32

La arquitectura MIPS32 dispone de 32 registros de propósito general de 32 bits cada uno. La siguiente tabla muestra la convención de nombres y el uso estándar de cada registro [3]:

Número	Nombre	Nombre Alternativo	Uso
0	\$zero	\$0	Siempre contiene el valor 0
1	\$at	\$1	Reservado para el ensamblador
2	\$v0	\$2	Valor de retorno de funciones
3	\$v1	\$3	Valor de retorno de funciones
4	\$a0	\$4	Argumento 1 para funciones

5	\$a1	\$5	Argumento 2 para funciones
6	\$a2	\$6	Argumento 3 para funciones
7	\$a3	\$7	Argumento 4 para funciones
8-15	\$t0-\$t7	\$8-\$15	Temporales (no preservados entre llamadas)
16-23	\$s0-\$s7	\$16-\$23	Registros preservados (callee-saved)
24-25	\$t8-\$t9	\$24-\$25	Temporales adicionales
26-27	\$k0-\$k1	\$26-\$27	Reservados para el kernel del SO
28	\$gp	\$28	Puntero al área global
29	\$sp	\$29	Puntero de pila (Stack Pointer)
30	\$fp	\$30	Puntero de frame (Frame Pointer)
31	\$ra	\$31	Dirección de retorno

Cuadro 1: Registros de propósito general MIPS32 [4]

3.1. Convenciones de Uso de Registros

- **\$zero**: Siempre lee como 0, las escrituras no tienen efecto
- **\$v0-\$v1**: Contienen el resultado de una función
- **\$a0-\$a3**: Se usan para pasar los primeros 4 argumentos a funciones
- **\$sp**: Apunta al tope de la pila (crece hacia direcciones menores)
- **\$ra**: Contiene la dirección de retorno al hacer `jal`

4. Conjunto de Instrucciones

4.1. Instrucciones Aritméticas

Las operaciones aritméticas básicas siguen el formato: `operación Rd, Rs, Rt` donde `Rd = Rs operación Rt` [2].

Listing 1: Ejemplos de instrucciones aritméticas

```

1 add $t0, $t1, $t2    # $t0 = $t1 + $t2
2 sub $t0, $t1, $t2    # $t0 = $t1 - $t2
3 addi $t0, $t1, 10    # $t0 = $t1 + 10
4 mul $t0, $t1, $t2    # $t0 = $t1 * $t2

```

Instrucción	Sintaxis	Descripción
ADD	add Rd, Rs, Rt	$Rd = Rs + Rt$ (con detección de overflow)
ADDI	addi Rt, Rs, Imm16	$Rt = Rs + Imm16$ (con overflow)
ADDU	addu Rd, Rs, Rt	$Rd = Rs + Rt$ (sin overflow)
ADDIU	addiu Rt, Rs, Imm16	$Rt = Rs + Imm16$ (sin overflow)
SUB	sub Rd, Rs, Rt	$Rd = Rs - Rt$ (con overflow)
SUBU	subu Rd, Rs, Rt	$Rd = Rs - Rt$ (sin overflow)
MUL	mul Rd, Rs, Rt	$Rd = Rs * Rt$
DIV	div Rs, Rt	$Lo = Rs / Rt, Hi = Rs \% Rt$

Cuadro 2: Instrucciones aritméticas MIPS32

4.2. Instrucciones Lógicas

Listing 2: Ejemplos de operaciones lógicas

```

1 and $t0, $t1, $t2      # $t0 = $t1 AND $t2
2 or  $t0, $t1, $t2      # $t0 = $t1 OR $t2
3 nor $t0, $t1, $t2      # $t0 = NOT($t1 OR $t2)
4 xor $t0, $t1, $t2      # $t0 = $t1 XOR $t2

```

Instrucción	Sintaxis	Descripción
AND	and Rd, Rs, Rt	$Rd = Rs \& Rt$
ANDI	andi Rt, Rs, Imm16	$Rt = Rs \& Imm16$
OR	or Rd, Rs, Rt	$Rd = Rs \mid Rt$
ORI	ori Rt, Rs, Imm16	$Rt = Rs \mid Imm16$
NOR	nor Rd, Rs, Rt	$Rd = \sim(Rs \mid Rt)$
XOR	xor Rd, Rs, Rt	$Rd = Rs \hat{Rt}$
XORI	xori Rt, Rs, Imm16	$Rt = Rs \hat{Imm16}$

Cuadro 3: Instrucciones lógicas MIPS32

4.3. Instrucciones de Desplazamiento

Instrucción	Sintaxis	Descripción
SLL	sll Rd, Rt, Imm5	$Rd = Rt \ll Imm5$ (desplazamiento logico izquierda)
SLLV	sllv Rd, Rt, Rs	$Rd = Rt \ll Rs$
SRL	srl Rd, Rt, Imm5	$Rd = Rt \gg Imm5$ (desplazamiento logico derecha, rellena con 0)
SRA	sra Rd, Rt, Imm5	$Rd = Rt \gg Imm5$ (desplazamiento aritmetico, rellena con MSB)

Cuadro 4: Instrucciones de desplazamiento

4.4. Instrucciones de Transferencia de Datos

Estas instrucciones permiten mover datos entre registros y memoria [2]:

Listing 3: Ejemplos de carga y almacenamiento

```

1 lw $t0, 0($s1)      # Carga palabra en $t0 desde Mem[$s1 + 0]
2 sw $t0, 4($s1)      # Almacena palabra en Mem[$s1 + 4] desde $t0
3 lb $t0, 0($s1)      # Carga byte (con extension de signo)
4 lbu $t0, 0($s1)     # Carga byte (sin extension de signo)
5 li $t0, 100         # Carga inmediata (pseudoinstruccion)
6 la $t0, etiqueta    # Carga direccion de etiqueta

```

Instruccion	Sintaxis	Descripcion
LW	lw Rt, Offset16(Rb)	Rt = Mem[Rb + Offset16] (palabra)
SW	sw Rt, Offset16(Rb)	Mem[Rb + Offset16] = Rt (palabra)
LB	lb Rt, Offset16(Rb)	Rt = Mem[Rb + Offset16] (byte con signo)
LBU	lbu Rt, Offset16(Rb)	Rt = Mem[Rb + Offset16] (byte sin signo)
LH	lh Rt, Offset16(Rb)	Rt = Mem[Rb + Offset16] (half-word con signo)
LUI	lui Rt, Imm16	Rt = Imm16 $\ll$ 16

Cuadro 5: Instrucciones de carga/almacenamiento

4.5. Instrucciones de Comparación y Salto

Las instrucciones de comparación (**slt** - Set on Less Than) establecen un registro a 1 si la condición se cumple, o a 0 en caso contrario [2].

Listing 4: Ejemplos de comparación y saltos

```

1 slt $t0, $t1, $t2    # $t0 = 1 si $t1 < $t2, sino 0
2 beq $t1, $t2, label  # Salta a label si $t1 == $t2
3 bne $t1, $t2, label  # Salta a label si $t1 != $t2
4 j label              # Salto incondicional (pseudoinstruccion)
5 jal funcion          # Salta a funcion y guarda retorno en $ra
6 jr $ra               # Retorna de funcio n

```

Instrucción	Sintaxis	Descripción
SLT	slt Rd, Rs, Rt	Rd = 1 si Rs < Rt, sino 0 (con signo)
SLTU	sltu Rd, Rs, Rt	Rd = 1 si Rs < Rt, sino 0 (sin signo)
SLTI	slti Rt, Rs, Imm16	Rt = 1 si Rs < Imm16, sino 0
BEQ	beq Rs, Rt, Label	Salta si Rs == Rt
BNE	bne Rs, Rt, Label	Salta si Rs != Rt
BGEZ	bgez Rs, Label	Salta si Rs ≥ 0
BLEZ	blez Rs, Label	Salta si Rs ≤ 0
BGTZ	bgtz Rs, Label	Salta si Rs > 0
BLTZ	bltz Rs, Label	Salta si Rs < 0

J	j Label	Salto incondicional a Label
JAL	jal Label	Salto y guarda retorno en \$ra
JR	jr Rs	Salto a dirección contenida en Rs

Cuadro 6: Instrucciones de comparación y salto

4.6. Instrucciones de Control del Sistema - Syscall

La instrucción `syscall` permite realizar llamadas al sistema operativo (o al simulador en MARS). El valor en `$v0` determina qué operación realizar [2]:

Código (\$v0)	Operación	Argumentos/Resultados
1	Imprimir entero	\$a0 = entero a imprimir
4	Imprimir cadena	\$a0 = dirección de cadena terminada en null
5	Leer entero	Resultado devuelto en \$v0
8	Leer cadena	\$a0 = buffer, \$a1 = longitud máxima
10	Salir del programa	-
11	Imprimir carácter	\$a0 = carácter a imprimir
12	Leer carácter	Resultado devuelto en \$v0

Cuadro 7: Syscalls útiles en MARS

Listing 5: Ejemplo de programa completo con syscalls

```

1 .data
2 mensaje: .asciiz "Hola_Mundo\n"
3 numero:  .word 42
4
5 .text
6 main:
7     # Imprimir cadena
8     li $v0, 4
9     la $a0, mensaje
10    syscall
11
12    # Imprimir entero
13    li $v0, 1
14    lw $a0, numero
15    syscall
16
17    # Salir
18    li $v0, 10
19    syscall

```

5. Estructura de un Programa en Ensamblador MIPS

Un programa típico en MIPS tiene dos secciones principales [2]:

Listing 6: Estructura básica de un programa MIPS

```
1 .data
2     # Variables globales y datos inicializados
3     mensaje: .asciiz "Ingrese un número:"
4     resultado: .word 0
5
6 .text
7     .globl main
8 main:
9     # C digo del programa
10
11     # Salida del programa
12     li $v0, 10
13     syscall
```

5.1. Directivas de Datos más Comunes

Directiva	Descripción
.byte valor	Almacena uno o más bytes
.half valor	Almacena half-words (16 bits)
.word valor	Almacena palabras (32 bits)
.asciiz "texto"	Almacena cadena terminada en null
.space N	Reserva N bytes sin inicializar

Cuadro 8: Directivas de datos en MIPS

6. Pseudoinstrucciones

MARS (al igual que otros ensambladores MIPS) soporta pseudoinstrucciones que facilitan la programación. El ensamblador las traduce automáticamente a instrucciones reales [2]:

Pseudoinstrucción	Equivalencia	Descripción
li Rd, Imm	addiu Rd, \$zero, Imm	Load Immediate
la Rd, Label	addiu Rd, \$zero, direccion	Load Address
move Rd, Rs	addiu Rd, Rs, 0	Copia registro
nop	sll \$zero, \$zero, 0	No operation
b Label	beq \$zero, \$zero, Label	Salto incondicional
blt Rs, Rt, Label	slt at, Rs, Rt; bne at, \$zero, Label	Branch if less than
neg Rd, Rs	sub Rd, \$zero, Rs	Negación

7. Modos de Direcccionamiento

MIPS32 soporta varios modos de direccionamiento [2]:

- **Direccionamiento por registro:** El operando está en un registro (ej. `add $t0, $t1, $t2`)
- **Direccionamiento inmediato:** El operando es una constante (ej. `addi $t0, $t1, 10`)
- **Direccionamiento base+desplazamiento:** Acceso a memoria usando registro base y offset (ej. `lw $t0, 8($s1)`)
- **Direccionamiento relativo a PC:** Usado por instrucciones de salto condicional (`beq`, `bne`)
- **Direccionamiento pseudo directo:** Usado por `j` y `jal`

8. Ejemplos Prácticos

8.1. Suma de dos números ingresados por el usuario

Listing 7: Programa para sumar dos números

```
1 .data
2     prompt1: .asciiz "Ingresa el primer numero: "
3     prompt2: .asciiz "Ingresa el segundo numero: "
4     result: .asciiz "La suma es: "
5
6 .text
7     .globl main
8 main:
9     # Solicitar primer numero
10    li $v0, 4
11    la $a0, prompt1
12    syscall
13
14    li $v0, 5
15    syscall
16    move $t0, $v0
17
18    # Solicitar segundo numero
19    li $v0, 4
```



```

20     la $a0, prompt2
21     syscall
22
23     li $v0, 5
24     syscall
25     move $t1, $v0
26
27     # Sumar
28     add $t2, $t0, $t1
29
30     # Mostrar resultado
31     li $v0, 4
32     la $a0, result
33     syscall
34
35     li $v0, 1
36     move $a0, $t2
37     syscall
38
39     # Salir
40     li $v0, 10
41     syscall

```

8.2. Bucle for para sumar números del 1 al N

Listing 8: Suma de 1 a N

```

1 .data
2     prompt: .asciiz "Ingrese N: "
3     result: .asciiz "La suma de 1 a N es: "
4
5 .text
6     .globl main
7 main:
8     li $v0, 4
9     la $a0, prompt
10    syscall
11
12    li $v0, 5
13    syscall
14    move $t0, $v0          # $t0 = N
15
16    li $t1, 0              # suma = 0
17    li $t2, 1              # contador = 1
18
19 bucle:
20    bgt $t2, $t0, fin      # if contador > N: salir

```

```

21      add $t1, $t1, $t2      # suma += contador
22      addi $t2, $t2, 1      # contador++
23      j bucle
24
25 fin:
26      li $v0, 4
27      la $a0, result
28      syscall
29
30      li $v0, 1
31      move $a0, $t1
32      syscall
33
34      li $v0, 10
35      syscall

```

9. Referencias

Referencias

- [1] Patterson, D. A., & Hennessy, J. L. (2012). *Computer Organization and Design: The Hardware/Software Interface*. 4th ed. Morgan Kaufmann.
- [2] UNSW Sydney. (2024). *MIPS Instruction Set Guide*. COMP1521 Course Materials.
- [3] SPIM Documentation. *MIPS Registers and Usage Convention*. Wheaton College.
- [4] MIPS Technologies. (2011). *MIPS32 74Kc Programming Manual*.